Database Users, Roles, Privileges, and Access Control

Group 5 (PUP-SPOT (System for Property Ownership Tracker))

Section	Contents
Cover and links	**GROUP 12: Amoranto, Denielle; Eurescial, Eunice; Francisco, Helix** **Project Assigned:** PUP-SPOT (System for Property Ownership Tracker) **Repository Link: https://github.com/grp05-dba/grp05_lostandfound** **Our Group Repository:**
Projects set up	**The grp05_lostandfound repository was used as the basis for implementing the database access-control plan. The project is a Lost and Found Management System designed to manage information about lost and found items, user accounts, and related records.** **The first step was to examine the existing project files and understand how the application connects to its database. The database configuration and existing SQL files were reviewed to identify the database name, tables, and other database objects that needed access restrictions.** **The database used for this activity was named [INSERT ACTUAL DATABASE NAME]. The existing database structure was retained, and the access-control implementation was performed separately to avoid changing the original system functionality.** **The following database tables were identified:** • **[ACTUAL TABLE NAME 1] – Stores information about [describe its purpose].** • **[ACTUAL TABLE NAME 2] – Stores information about [describe its purpose].** • **[ACTUAL TABLE NAME 3] – Stores information about [describe its purpose].** • **[ACTUAL TABLE NAME 4] – Stores information about [describe its purpose].** **After reviewing the database structure, a separate folder named `Activity-4` was prepared to organize the SQL scripts and access-control documentation.**

	The following files were created: 1. `01_create_roles.sql` 2. `02_grant_privileges.sql` 3. `03_revoke_privileges.sql` 4. `04_test_privileges.sql` 5. `access_control_plan.md` **These files were organized to make the implementation easier to understand, execute, and maintain.**
Access-control plan	**The access-control plan was created to determine which database operations should be allowed for each role. The implementation follows the Principle of Least Privilege, meaning that each role should only receive the permissions necessary for its responsibilities.** **Three PostgreSQL group roles were established.** ### Administrator Role (lf_admin) **The administrator role is responsible for managing and maintaining the database. It receives full table privileges on the existing tables within the public schema.** **Its responsibilities include viewing database records, inserting new records, updating existing information, and deleting records when necessary.** ### Staff Role (lf_staff) **The staff role is intended for users who manage lost and found records. It allows the retrieval, addition, and updating of database records.** **Its responsibilities include viewing lost and found information, adding new records, and updating existing records.** **The staff role is restricted from deleting records to help prevent unauthorized data removal.** ### Viewer Role (lf_viewer) **The viewer role is intended for users who only need to view database information. It receives SELECT privileges but cannot insert, update, or**

<table>
<tr>
<td></td>
<td>
delete records.

Its purpose is to allow authorized users to view lost and found information without making unauthorized changes.

Access-Control Matrix

<table>
<tr><th>Database Operations</th><th>Administrator</th><th>Staff</th><th>Viewer</th></tr>
<tr><td>SELECT</td><td>Allowed</td><td>Allowed</td><td>Allowed</td></tr>
<tr><td>INSERT</td><td>Allowed</td><td>Allowed</td><td>Denied</td></tr>
<tr><td>UPDATE</td><td>Allowed</td><td>Allowed</td><td>Denied</td></tr>
<tr><td>DELETE</td><td>Allowed</td><td>Denied</td><td>Denied</td></tr>
</table>

Database Users and Role Assignments

The following database login accounts were prepared:

<table>
<tr><th>Database User</th><th>Assigned Role</th></tr>
<tr><td>lf_admin_user</td><td>lf_admin</td></tr>
<tr><td>lf_staff_user</td><td>lf_staff</td></tr>
<tr><td>lf_viewer_user</td><td>lf_viewer</td></tr>
</table>

The three group roles were created with the NOLOGIN attribute. This means they function as permission groups and must be assigned to actual login accounts before individual users or application accounts can use their permissions.
</td>
</tr>
<tr>
<td>SQL scripts summary</td>
<td>Four main SQL implementation and verification scripts were prepared to implement and check the access-control configuration.</td>
</tr>
</table>

	File 1: 01_create_roles.sql **The first SQL script was used to create the three PostgreSQL group roles and their corresponding login accounts.** **The following group roles were created:** • `lf_admin` • `lf_staff` • `lf_viewer` **The following database login accounts were also created:** • `lf_admin_user` • `lf_staff_user` • `lf_viewer_user` **Each login account was assigned to its corresponding group role using PostgreSQL GRANT statements.** **This allows the database users to inherit the permissions assigned to their respective roles.** **File 2: 02_grant_privileges.sql** **The second SQL script was used to assign the required database permissions to each role.** **First, the three roles were given USAGE permission on the public schema.** **The administrator role received ALL PRIVILEGES on all existing tables and sequences in the public schema.** **The staff role received SELECT, INSERT, and UPDATE permissions on all existing tables in the public schema.** **The viewer role received SELECT privileges only on all existing tables in the public schema.** **The use of ON ALL TABLES IN SCHEMA public allows the permissions to be assigned without manually specifying each existing table name.**

	File 3: 03_revoke_privileges.sql **The third SQL script was used to remove privileges that should not be available to restricted roles.** **The staff role was restricted from performing DELETE operations.** **The viewer role was restricted from performing INSERT, UPDATE, and DELETE operations.** **The script also ensures that staff retains SELECT, INSERT, and UPDATE permissions, while viewers retain SELECT permissions.** **This helps maintain the Principle of Least Privilege by removing unnecessary access rights.** **File 4: 04_test_privileges.sql** **The fourth SQL script was used to check whether the assigned permissions were correct.** **The script contains queries that display the created roles, list their table privileges, and test specific permissions using PostgreSQL's `has_table_privilege()` function.** **The verification focused on checking whether:** ● **The administrator can perform SELECT operations.** ● **The administrator can perform INSERT operations.** ● **The administrator can perform UPDATE operations.** ● **The administrator can perform DELETE operations.** ● **The staff role can view, insert, and update records.** ● **The staff role cannot delete records.** ● **The viewer role can select records.** ● **The viewer role cannot insert records.** ● **The viewer role cannot update or delete records.** **These checks help confirm that each role follows the intended access-control plan.**
Verification results	**After preparing the SQL scripts, they were executed in the PostgreSQL Query Tool through pgAdmin 4.**

	The scripts were executed in the correct order to avoid permission-related errors. **First Verification: Role Creation** **The first script was executed to create the three group roles and their corresponding login accounts.** **A verification query was used to display the created roles from the PostgreSQL system catalog.** **Expected result: All three group roles and their login accounts should appear in the query results.** **Second Verification: Privilege Assignment** **The second script was executed to assign the required schema, table, and sequence privileges.** **The execution messages were checked to determine whether the commands completed successfully.** **Expected result: Each role should receive the permissions defined in the access-control plan.** **Third Verification: Privilege Restrictions** **The third script was executed to remove unnecessary privileges from restricted roles.** **Expected result:** ● **The staff role should not have DELETE privileges.** ● **The viewer role should not have INSERT, UPDATE, or DELETE privileges.** ● **The viewer role should retain SELECT privileges.** **Fourth Verification: Permission Checking** **The fourth script was executed to verify the permissions using PostgreSQL queries.** **The expected results were:**

	Verification	Expected Result
	Administrator can SELECT records	TRUE
	Administrator can INSERT records	TRUE
	Administrator can UPDATE records	TRUE
	Administrator can DELETE records	TRUE
	Staff can SELECT records	TRUE
	Staff can INSERT records	TRUE
	Staff can UPDATE records	TRUE
	Staff can DELETE records	FALSE
	Viewer can SELECT records	TRUE
	Viewer can INSERT records	FALSE

	Viewer can UPDATE records FALSE **Viewer can DELETE records FALSE** **The expected results demonstrate that the administrator and staff roles have their intended database permissions, while the viewer role is restricted to viewing records.** **Note: These are expected results, not confirmed execution results. Actual results must be recorded after running the scripts and testing the permissions using the assigned roles.** --- ## Screenshots **The following screenshots should be included as supporting evidence:** - **Screenshot 1: `G5_create_roles.png`** - **Screenshot 2: `G5_grant_privileges.png`** - **Screenshot 3: `G5_revoke_privileges.png`** - **Screenshot 4: `G5_test_privileges.png`** **The screenshots serve as supporting evidence of the SQL execution and verification process.**
Issues and fixes	**During the preparation and implementation of the access-control configuration, the following possible issues were considered and addressed.** ### Database Name Mismatch **The SQL scripts must be executed inside the correct database. The actual Lost and Found database name must be used consistently to avoid applying the configuration to the wrong database.** ### Existing Database Roles **Running role creation commands repeatedly may cause errors if the roles already exist. The first SQL script should check whether each role**

	exists before creating it. **Incorrect Execution Order** **Granting permissions before creating the roles can result in errors. To prevent this, the scripts were executed in order, starting with role creation, followed by privilege assignment, privilege restrictions, and verification.** **Insufficient Database Permissions** **Some PostgreSQL commands require appropriate administrative privileges. The scripts should be executed using an authorized PostgreSQL administrator account.** **Incorrect Table Names** **Granting privileges to tables that do not exist can result in relation errors.** **Initially, the use of `public.example_table` caused an error because the placeholder table did not exist in the actual database.** **To address this issue, the privilege assignment script was updated to use:** `ON ALL TABLES IN SCHEMA public` **This allows PostgreSQL to apply the specified privileges to all existing tables in the public schema without manually entering individual table names.** **Future Database Objects** **Newly created tables and sequences may not automatically receive the same permissions as existing objects. Default privileges should be configured for the correct object owner when necessary.** **Viewer Permission Restrictions** **The viewer role should not be able to modify database records. The verification script checks that SELECT is allowed while INSERT, UPDATE, and DELETE are denied.**

Feedback to owners	**The Lost and Found Management System provides database functionality for managing lost and found records. However, implementing Role-Based Access Control can further improve the security and organization of database operations.** **The following recommendations are provided to the project owners:** ● **Maintain separate database roles for administrators, staff, and viewers.** ● **Avoid using administrator-level database credentials for regular application activities.** ● **Assign actual login accounts to the appropriate group roles.** ● **Use strong passwords and protect database credentials.** ● **Review database permissions whenever new tables or features are added.** ● **Restrict access to sensitive user information and records.** ● **Regularly check database roles and privileges to prevent unnecessary access.** ● **Use read-only accounts for users who only need to view database information.** **These recommendations can help improve the security and maintainability of the Lost and Found Management System.**
AI usage declaration	**Artificial Intelligence (AI) tools were used as a supplementary resource during the activity to assist in understanding PostgreSQL role creation, database privileges, SQL syntax, and documentation preparation.** **AI assistance was used to help organize the SQL scripts, explain the purpose of each access-control role, and identify possible implementation issues and their corresponding solutions.** **The SQL scripts were reviewed and executed using PostgreSQL through pgAdmin 4. The student remains responsible for checking the correctness of the implementation, understanding the assigned permissions, and verifying the actual execution results.**